

مستندات آموزشی: اتصال C++ به پایگاه‌های داده MySQL و SQLite

مقدمه و معماری ارتباط با پایگاه داده

در پیاده‌سازی این مینی پروژه، سامانه باید تمامی داده‌ها را در پایگاه داده ذخیره کند و به نحوی طراحی شود که تعامل مستقیم بخش‌های مختلف برنامه با کوئری‌های SQL وجود نداشته باشد تا از پراکندگی کدهای پایگاه داده و سختی نگهداری جلوگیری شود. برای این منظور، باید یک یا چند کلاس واسط مانند کلاس Data ObjectAccess طراحی شود که نقش پل بین پایگاه داده و منطق برنامه را ایفا کند. این کلاس‌های واسط مسئول اجرای کوئری‌ها، خواندن داده‌ها و برگرداندن اطلاعات به شکل آبجکت‌های مناسب به بخش‌های دیگر برنامه هستند. کلاس‌های واسط باید به گونه‌ای طراحی شوند که عملیات CRUD (ساخت، خواندن، به‌روزرسانی، حذف) را به طور کامل پوشش دهند. در این مستند، نحوه انجام این عملیات پایه در دو سیستم محبوب MySQL و SQLite شرح داده شده است تا دسترسی غیرمستقیم از طریق کلاس‌های واسط به درستی محقق گردد.

بخش اول: تعامل با پایگاه داده SQLite

پایگاه داده SQLite یک انتخاب بسیار مناسب برای برنامه‌های محلی و بدون نیاز به سرور است. تمام داده‌ها در یک فایل متنی مجزا ذخیره می‌شوند که جابه‌جایی و تست برنامه را بسیار ساده می‌کند. برای استفاده از این کتابخانه در زبان سی‌پلاس‌پلاس، ابتدا باید فایل هدر مربوطه را به برنامه اضافه کرد.

عملیات اتصال به پایگاه داده

برای شروع کار، نیاز به ایجاد یک اشاره‌گر از نوع sqlite3 داریم که نمایانگر اتصال ما به فایل پایگاه داده است. با استفاده از تابع sqlite3_open این اتصال برقرار می‌شود. در صورتی که فایل وجود نداشته باشد، این تابع به صورت خودکار آن را ایجاد می‌کند. پس از اتمام کار نیز همواره باید با استفاده از تابع sqlite3_close این اتصال را با موفقیت بست تا از نشت حافظه جلوگیری شده و برنامه از نظر مدیریت حافظه ایمن باقی بماند.

```
#include <sqlite3.h>
#include <iostream>

sqlite3* db;
int exit = sqlite3_open("restaurant_app.db", &db);
if (exit) {
    std::cerr << "Error open DB: " << sqlite3_errmsg(db) << std::endl;
} else {
    std::cout << "Opened Database Successfully!" << std::endl;
}: sqlite3_close(db);
```

عملیات ایجاد یا افزودن داده (Create)

برای افزودن داده‌های جدید مانند ثبت یک رستوران یا آیتم منو، از دستور INSERT در زبان SQL استفاده می‌شود. اجرای این دسته از دستورات در SQLite توسط تابع sqlite3_exec انجام می‌پذیرد. این تابع کوئری متنی را دریافت کرده و آن را روی اتصال فعال اجرا می‌کند. در صورت بروز هرگونه خطا، پیام مربوطه در متغیری که به عنوان آرگومان آخر ارسال شده است، قرار می‌گیرد تا خطاهای احتمالی کنترل شده باشند.

```
std::string sql = "INSERT INTO RESTAURANT (ID, NAME, STATUS) VALUES
(1, 'Pizza Hut', 'ACTIVE');";
char* messageError;
int exit = sqlite3_exec(db, sql.c_str(), NULL, 0, &messageError);
if (exit != SQLITE_OK) {
    std::cerr << "Error Insert: " << messageError << std::endl;
    sqlite3_free(messageError);
} else {
    std::cout << "Records created successfully!" << std::endl;
}
```

عملیات خواندن داده‌ها (Read)

بازیابی اطلاعات حساس‌ترین بخش کار با SQLite است، زیرا برای پردازش رکوردهای بازگشتی به یک تابع بازخورد یا Callback نیاز داریم. زمانی که دستور SELECT را با تابع sqlite3_exec اجرا می‌کنیم، به ازای هر رکورد یافت شده، تابع بازخورد فراخوانی می‌شود و ستون‌های آن رکورد را در اختیار ما قرار می‌دهد. این مقادیر باید در کلاس‌های دامنه سیستم کپسوله شده و بازگردانده شوند.

```
int callback(void* data, int argc, char** argv, char** azColName) {
    for (int i = 0; i < argc; i++) {
        std::cout << azColName[i] << " = " << (argv[i] ? argv[i] :
"NULL") << std::endl;
    }
    std::cout << "-----" << std::endl;
    return 0;
}
```

```
std::string sql_select = "SELECT * FROM RESTAURANT;";
char* messageError;
int exit = sqlite3_exec(db, sql_select.c_str(), callback, NULL,
&messageError);
```

عملیات بروزرسانی و حذف (Delete & Update)

فرآیند بروزرسانی اطلاعات موجود مانند تغییر وضعیت یک سفارش به حالت ارسال شده، و همچنین حذف رکوردها کاملاً مشابه عملیات ایجاد است. کافی است کوئری مربوط به دستورات UPDATE یا DELETE را در قالب یک رشته متنی آماده کرده و آن را به تابع `sqlite3_exec` ارسال کنید. این فرآیندها به سیستم اجازه می‌دهند تا عملیات ویرایش و حذف در منوی مدیریتی به درستی به پایگاه داده منتقل شوند.

```
std::string sql_update = "UPDATE RESTAURANT set STATUS = 'INACTIVE'
where ID=1;";
sqlite3_exec(db, sql_update.c_str(), NULL, 0, &messageError);
```

```
std::string sql_delete = "DELETE from RESTAURANT where ID=1;";
sqlite3_exec(db, sql_delete.c_str(), NULL, 0, &messageError);
```

بخش دوم: تعامل با پایگاه داده MySQL

پایگاه داده MySQL یک سیستم مدیریت پایگاه داده رابطه‌ای مبتنی بر سرور است. برای اتصال برنامه سی‌پلاس‌پلاس به این سرور، نیاز به استفاده از کتابخانه رسمی `mysqlclient` است. رویکرد کار با این سیستم نیازمند مدیریت متمرکزتر روی نتایج بازگشتی در حافظه است.

برقراری ارتباط با سرور MySQL

جهت اتصال به سرور، در گام اول یک شیء از نوع MySQL مقداردهی اولیه می‌شود. سپس تابع `mysql_real_connect` با دریافت مشخصات سرور، نام کاربری، رمز عبور و نام پایگاه داده، اتصال را نهایی می‌کند. مدیریت صحیح این شیء و آزادسازی آن با دستور `mysql_close` در انتهای برنامه الزامی است.

```
#include <mysql/mysql.h>
#include <iostream>

MYSQL* conn = mysql_init(NULL);
if (conn == NULL) {
    std::cerr << "mysql_init() failed" << std::endl;
}

if (mysql_real_connect(conn, "localhost", "root", "password",
"delivery_db", 0, NULL, 0) == NULL) {
    std::cerr << "mysql_real_connect() failed: " << mysql_error(conn)
<< std::endl;
} else {
    std::cout << "Connected to MySQL successfully!" << std::endl;
}
```

```
mysql_close(conn);
```

اجرای عملیات پایه (Insert, Update, Delete)

اجرای دستوراتی که نتیجه بازگشتی ساختاریافته به شکل جدول ندارند، از طریق تابع `mysql_query` انجام می‌شود. این تابع رشته حاوی دستور SQL را دریافت کرده و در صورت اجرای موفقیت‌آمیز، مقدار صفر را برمی‌گرداند. این ساختار در متدهای کلاس‌های DAO برای ثبت یا ویرایش اطلاعات به کار گرفته می‌شود.

```
std::string query = "INSERT INTO USERS (USERNAME, ROLE) VALUES  
( 'admin', 'MANAGER' )";  
if (mysql_query(conn, query.c_str())) {  
    std::cerr << "INSERT failed. Error: " << mysql_error(conn) <<  
std::endl;  
}
```

عملیات استخراج اطلاعات (Read)

پس از اجرای دستور SELECT توسط `mysql_query`، نتایج موقتاً در سرور باقی می‌مانند. برای انتقال این داده‌ها به حافظه برنامه سی‌پلاس‌پلاس، باید از تابع `mysql_store_result` استفاده کرد. خروجی این تابع یک ساختار داده است که می‌توان با استفاده از حلقه و تابع `mysql_fetch_row`، رکورد به رکورد روی آن پیمایش کرد و مقادیر ستون‌ها را استخراج نمود. در نهایت نیز حتماً باید فضای اشغال شده توسط این نتایج را با دستور `mysql_free_result` آزاد کرد تا حافظه به درستی بازگردانده شود.

```
if (mysql_query(conn, "SELECT * FROM RESTAURANT")) {  
    std::cerr << "SELECT failed. Error: " << mysql_error(conn) <<  
std::endl;  
} else {  
    MYSQL_RES* result = mysql_store_result(conn);  
    if (result == NULL) {  
        std::cerr << "mysql_store_result() failed." << std::endl;  
    } else {  
        int num_fields = mysql_num_fields(result);  
        MYSQL_ROW row;  
        while ((row = mysql_fetch_row(result))) {  
            for(int i = 0; i < num_fields; i++) {  
                std::cout << (row[i] ? row[i] : "NULL") << " ";  
            }  
            std::cout << std::endl;  
        }  
        mysql_free_result(result);  
    }
```

}
}